

Document 13 — Testing, Evaluation & Observability

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[12_production_engineering](#)

How to Read & Practice This Document

- **What:** proving your system works, and catching it when it stops working.
- **Why:** systems that aren't always the same break silently without this — you won't notice something got worse until a user does, and by then it's a real problem, not just a code review comment.
- **When:** before you'd ever trust a prompt or model change in production; really, before you'd trust your own Project 4 as "finished."
- **How to practice:** 1. Read the OpenAI Evals README and one example test before writing your own — copy the *shape* of it, not the content. 2. Do the **Basic/Intermediate** tests closed-book where you can — these are normal pytest, no LLM involved. 3. Do the **Real-world** LLM-as-judge exercise, writing your scoring rules before you look at any output. 4. Do the **Edge case/Failure** exercises for real — make a prompt worse on purpose and confirm your test suite actually catches it. If it doesn't, that's the real lesson. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, show a real score drop from a real, on-purpose problem you caused. If you haven't made one happen, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

By this point, Project 4 (your multi-agent system) runs. But "runs" and "works" aren't the same thing — an LLM system can run without errors while quietly giving worse answers than it did last week, and nothing on your screen will tell you.

First, you need a way to check correctness that doesn't assume the model says the exact same words every time. Two answers can use completely different wording and both be right — so instead of checking "is this exactly what I expected," you check "does this have the *properties* a correct answer must have." That's the mindset shift the **testing layers** are built on: plain, exact tests for the parts that don't touch a model at all (chunking, tool logic, routing), and property-based checks for the parts that do.

Second, once you can check correctness once, you need to check it *again and again* — every time you touch a prompt or swap a model. This is **regression testing**: a fixed set of tasks with fixed pass/fail rules, run automatically, so a change that quietly makes things worse gets caught by your test suite instead of by a user.

Third, some of what makes an answer "good" can't be checked by ordinary code at all — is this summary actually good, not just non-empty? That's what **LLM-as-a-judge** is for: a second model call, scoring the first model's answer against rules you wrote down yourself. The judge is not a neutral referee — it's a piece of software you built, with a prompt you have to get right, exactly like any other prompt.

Fourth, once a system is live, you can't watch every run happen — so you need a **record** of what happened in each one: the input, every tool call, the final output, timing, cost. Without that record, a bad answer is just a mystery you can't go back and investigate.

That's the whole story: testing layers give you a way to check correctness without assuming exact wording. A judge extends that check into "quality," not just "shape." A regression suite makes sure a change never quietly makes things worse without you noticing. And run records let you understand, after the fact, exactly what happened when something goes wrong. The **Build Task** below asks you to put the first three of these together into one real, runnable test suite for Project 4 — the thing that actually tells you whether you can trust a change before you ship it.

Core Concepts (read this first — everything you need is here)

Topics on this page: [Why testing something that isn't always the same needs a different mindset](#) · [The testing layers, and what each one actually checks](#) · [Regression testing for prompts](#) · [Test sets and LLM-as-a-judge](#) · [Watching your system: what to record for each run](#)

Why testing something that isn't always the same needs a different mindset

Normal unit tests check exact equality: given this input, the output must be exactly that. An LLM given the same input can word its answer differently each time, while still being just as correct — checking for the exact same words makes tests fail for the wrong reason (the system is actually fine, the check is just too strict). **Why this needs a mindset shift:** you're not checking "is the output identical," you're checking "does the output have the *qualities* a correct answer must have" — has the right fact, is valid JSON, scores above some quality bar, calls the right tool. **How this changes**

what you write: checks based on properties and score thresholds, instead of exact-match checks, for anything touching the model directly.

The testing layers, and what each one actually checks

- **Unit tests** — pure logic, no LLM involved (chunking functions, a tool's own math, a routing check's logic). Fast, exact, always the same — the base everything else stands on.
- **Integration tests** — several pieces together (a tool call flowing through your checking layer), often still fine to fake the LLM part to avoid real API calls.
- **API tests** — your FastAPI routes, checking request/response shapes and status codes, separate from what the agent does inside.
- **Tool tests** — one tool's behavior on its own, faking the model's decision to call it.
- **Agent/workflow tests** — the full loop or graph, checking it reaches the right final states for given inputs — this is where "not always the same" starts to matter, and property-based checks replace exact ones.
- **End-to-end tests** — the whole system, the way a user would actually experience it. **Why the layers matter:** a failure at the unit level points exactly at broken logic; a failure only visible end-to-end tells you *something's* wrong, but not *where* — you want failures caught as low in this list as possible, because lower-layer failures are cheaper and faster to figure out.

Regression testing for prompts

A prompt or model-version change can quietly make things worse on cases that weren't part of whatever quick testing you did while making the change — "quietly," because the system still runs with no errors, it just answers worse. **Why a fixed, repeatable test suite (below) is the real defense:** without one, you only find out about a regression when a user notices — which costs a lot more than catching it before you ship the change.

Test sets and LLM-as-a-judge

A **test set** is a fixed group of inputs with clear pass/fail rules, run automatically — the testing version of a unit-test suite, but for behavior you can't just check with `==`. Where rules can't be checked by code (is this summary actually *good*, not just non-empty), **LLM-as-a-judge** uses a second model call to score the first model's answer against clear rules you write into the judge's prompt. **Why the judge's prompt itself needs the same care as any other prompt:** a badly written judge (one that rewards long answers no matter how correct they are) will happily approve bad answers, giving you false confidence — the judge isn't some neutral referee, it's a piece you built, and it can be wrong, exactly like anything else you build.

Watching your system: what to record for each run

For a production LLM system, "what actually happened in this one run" needs to be something you can put back together after the fact — which means recording, for every run: the full input, every

step and tool call along the way and what it returned, the final output, timing, and token/cost usage.

Why this is different from normal app logging: an LLM system's failures are often *quality* failures (a wrong but believable-sounding answer), not crashes — without a recorded run, there's no way to look back and understand why one specific bad answer happened, since the model itself can't reliably explain its own reasoning after the fact.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [pytest documentation](#) — fixtures, parametrize; your test tool from here on.
- [OpenAI Evals \(GitHub\)](#) — a real framework for testing LLMs; read the README and one example.
- [LangSmith documentation](#) — run-tracking and testing, specifically for LangChain/LangGraph apps.

Practice Exercises

Setup for this document's practice code: work inside

`13_testing_evaluation_observability/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install pytest`.

How to run each exercise: save it as its own small script — `practice_basic.py`, `practice_intermediate.py`, and so on, matching the levels below — and run it directly: `python practice_basic.py`. Keep each one runnable on its own; don't chain them into one file.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — a normal, exact test

- **What:** a normal pytest test for `08_rag's chunking.py` (pure logic, no LLM call, exact checks).
- **Why:** this is the foundation layer — get comfortable with ordinary, always-the-same testing before adding the harder, non-deterministic layer on top.
- **When you'll hit this for real:** every pure-logic function in every project — tools' internal math, routing functions, chunking, anything with no model call inside it.
- **How to code it:** `def test_chunk_by_paragraph(): chunks = chunk_by_paragraph(sample_text); assert len(chunks) == 3` — an exact, ordinary `assert`, run with `pytest`.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — test a tool without calling the LLM

- **What:** a test for `06_tools_function_calling`'s tool functions, faking the LLM entirely.
- **Why:** this separates "does my tool's own logic work" from "does the model choose to call it" — two completely different questions that should be tested separately.
- **When you'll hit this for real:** any tool with real logic worth testing on its own, independent of whether the model ever calls it correctly.
- **How to code it:** call your tool function directly with hardcoded arguments (skip the model call entirely), and assert on its return value.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — score an answer that isn't exact

- **What:** a test for an LLM answer that can't be checked with `==` — scoring rules written first, then the test built.
- **Why:** this is the actual skill this whole document is about — everything before this exercise was preparation for this one.
- **When you'll hit this for real:** this document's own Build Task, testing Project 4's real output quality.
- **How to code it:** write down 2-3 concrete pass/fail rules first (e.g. "mentions the correct category," "under 100 words"), then write a test that checks the real output against those rules, not against one exact string.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — fix a flaky test

- **What:** a test that fails randomly because it checks exact LLM wording, rewritten to check a property instead.
- **Why:** a test that fails randomly gets ignored by everyone eventually — "oh, that one's just flaky" — which means it stops protecting you at all.
- **When you'll hit this for real:** the first time you write `assert response == "expected text"` against a real LLM call and it fails on a re-run with an equally correct answer.
- **How to code it:** take a test asserting exact text equality, and rewrite it to assert a property instead — `assert "billing" in response.lower()`, or `assert is_valid_json(response)`.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — prove the suite actually catches a regression

- **What:** make a prompt worse on purpose, and confirm your test suite actually catches the drop. If it doesn't, find the hole.
- **Why:** an eval suite you've never watched fail is a suite you don't actually know works — this is the one test of the tests themselves.

- **When you'll hit this for real:** this document's own Build Task, and every time you'd otherwise just trust a green checkmark without questioning it.
- **How to code it:** temporarily delete a key instruction from a working prompt, re-run your test suite, and confirm the score actually drops. If it doesn't, your test rules are too loose — tighten them until it does.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Test Suite for Project 4

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#) **Goal:** a runnable test suite for Project 4 (the multi-agent system) that would actually catch a real regression.

Requirements:

- A fixed set of test tasks (10+) with clear pass/fail or scored rules — written down before you run anything against them.
- At least one normal, always-the-same test layer (tool logic, chunking, routing functions) with exact checks.
- At least one LLM-as-judge test for final answer quality, with the judge's own prompt written down and reviewed (not just "ask GPT if this is good").
- A regression check: run the suite, make one prompt worse on purpose, run it again, confirm the score drops.

Inputs: the fixed set of tasks; the current Project 4 pipeline (and later, a version you've deliberately made worse).

Outputs: a pass/fail or number for each task, and one overall score.

Constraints: the suite must run without you watching it (`pytest` or a script) — no eyeballing needed to get a result.

Suggested files:

```
13_testing_evaluation_observability/
├─ eval_suite/
│   ├─ tasks.py           (the fixed set of test tasks)
│   ├─ judge.py           (LLM-as-judge scoring)
│   └─ run_eval.py
└─ test_unit_layer.py     (normal tests, like tool logic, chunking)
```

Functions/Components to build:

- `tasks.py` → the fixed list of test tasks with what a good answer looks like

- `judge.py` → `score_output(task, output) -> EvalResult`, using an LLM judge with a saved, versioned prompt
- `run_eval.py` → runs the full suite, prints/saves an overall report

Expected Behavior

- The suite runs without you watching, and gives a clear pass/fail plus score report.
- A prompt made worse on purpose causes a real, measurable score drop when you run the suite again.
- Unit-level tests fail fast, and point at the exact spot — not a vague "something's wrong" — when pure logic breaks.

Test Cases

Scenario	Expected
Normal pipeline, full suite	A passing score above your chosen bar
Prompt made worse on purpose, full suite	A real score drop, ideally below the bar
Tool logic broken directly (unit layer)	The one specific unit test fails, pointing at the exact function
Judge given an obviously bad answer	Judge scores it low, doesn't just approve it

Break-It / Debug Preview

- A test that fails randomly because it checks for exact LLM wording.
- A judge prompt that favors long answers no matter how correct they are.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- How you test something with no single correct answer · LLM-as-judge pitfalls · what you'd watch in production to catch a quiet quality drop · unit vs. integration vs. eval testing for agents.

Move On When

You have a runnable test suite for Project 4 that would actually catch a regression if the prompts were sabotaged. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"